

Data Science Capstone, Choose Your Own

hbmaclean

2025-07-05

Introduction

For this Data Science capstone project, I have chosen to analyze mushroom data from UCI.edu - to attempt to predict whether a mushroom is poisonous or edible, given color, odor, cap, stalk, habitat, and other descriptions.

From Audobon<sic> Society Field Guide; mushrooms described in terms of physical characteristics; classification: poisonous or edible This data set includes descriptions of hypothetical samples corresponding to 23 species of gilled mushrooms in the Agaricus and Lepiota Family (pp. 500-525). Each species is identified as definitely edible, definitely poisonous, or of unknown edibility and not recommended. This latter class was combined with the poisonous one. The Guide clearly states that there is no simple rule for determining the edibility of a mushroom; no rule like “leaflets three, let it be” for Poisonous Oak and Ivy.

I have no expertise in the area, but know enough about mushrooms to be intrigued. In addition to their culinary and medicinal uses, I know mushrooms are critical for breaking down organic material; are used in soil and water remediation projects like oil spills; and have numerous symbiotic relationships, including with algae (together composing lichen) and Indian pipes (a wildflower that does not photosynthesize.)

I did harvest and eat mushrooms once - a large, textbook cluster of *Grifola frondosa*, Hen-of-the-woods, growing at the base of an oak tree in my yard. Once dried, it added a nice earthy flavor to soups and other dishes.

Data Creation

This downloads the zip, extracts the data file, reads and parses the lines into a `data.frame` and adds column names.

```
# Download mushroom.zip from UCI.edu and render it as a data.frame
fetch_data <- function(verbose=FALSE) {
  options(timeout = 120)
  tmp  <- tempfile()
  zip  <- "https://archive.ics.uci.edu/static/public/73/mushroom.zip"

  download.file(zip, tmp, quiet = !verbose)
  data <- as.data.frame(
    str_split(
      read_lines(
        unz(tmp, "agaricus-lepiota.data")
      ),
      ",",
      simplify=TRUE
    )
  )

  colnames(data) <- c(
    "poisonous", "cap_shape", "cap_surface", "cap_color",
    "bruises", "odor", "gill_attachment", "gill_spacing",
    "gill_size", "gill_color", "stalk_shape", "stalk_root",
    "stalk_surface_above_ring", "stalk_surface_below_ring",
    "stalk_color_above_ring", "stalk_color_below_ring",
    "veil_type", "veil_color", "ring_number", "ring_type",
    "spore_print_color", "population", "habitat"
  )
  return(data)
}

mushroom_df <- fetch_data()
mushroom_df |> head() |> select(poisonous, cap_shape, gill_color, odor, habitat)
```

	poisonous	cap_shape	gill_color	odor	habitat
## 1	p	x	k	p	u
## 2	e	x	k	a	g
## 3	e	b	n	l	m
## 4	p	x	n	p	u
## 5	e	x	k	n	g
## 6	e	x	n	a	g

Data Pre-Processing

For much analysis, numeric data is supportive of more tools. But for plotting, I'll want human-friendly data.

Making numeric is a simple matter of matching `letters`, with special attention toward the the first column, the outcome.

```
# Convert single-letter abbreviations to numbers for all but first column.  
# For the first column, convert 'p' (poisonous) to 1, otherwise 0.  
# Some values are '?', which obviously do not map to letters and are coalesced to 0  
make_numeric <- function(data) {  
  data |>  
    rowwise() |>  
    mutate(across(-c(1), ~ coalesce(match(., letters), 0))) |>  
    mutate(poisonous = ifelse(poisonous == 'p', 1, 0)) |>  
    ungroup()  
}  
  
numeric_df <- make_numeric(mushroom_df)  
numeric_df |> head() |> select(poisonous, cap_shape, gill_color, odor, habitat)
```

```
## # A tibble: 6 x 5  
##   poisonous cap_shape gill_color odor habitat  
##       <dbl>    <dbl>    <dbl> <dbl>    <dbl>  
## 1         1        24         11    16        21  
## 2         0        24         11     1         7  
## 3         0         2         14    12        13  
## 4         1        24         14    16        21  
## 5         0        24         11    14         7  
## 6         0        24         14     1         7
```

To make the data human-friendly, there is a dictionary of terms provided on the website. I convert that to a hash lookup table:

```
# Map letters to their definitions, provided on website
make_legible <- function(data) {
  library(hash)

  # colors and surfaces are used several times
  colors <- hash(
    keys   = c('k', 'n', 'b', 'h', 'g', 'r', 'o', 'p', 'u', 'e', 'w', 'y', 'c'),
    values = c('black', 'brown', 'buff', 'chocolate', 'gray', 'green', 'orange',
               'pink', 'purple', 'red', 'white', 'yellow', 'cinnamon')
  )

  surfaces <- hash(
    keys   = c('f', 'y', 'k', 's', 'g'),
    values = c('fibrous', 'scaly', 'silky', 'smooth', 'grooves')
  )

  values_map <- hash(
    keys   = names(data),
    values = c(
      hash(
        keys   = c('p', 'e'),
        values = c(1, 0)
      ),
      hash(
        keys   = c('b', 'c', 'x', 'f', 'k', 's'),
        values = c('bell', 'conical', 'convex', 'flat', 'knobbed', 'sunken')
      ),
      surfaces,
      colors,
      hash(
        keys   = c('t', 'f'),
        values = c('bruises', 'no')
      ),
      hash(
        keys   = c('a', 'l', 'c', 'y', 'f', 'm', 'n', 'p', 's'),
        values = c('almond', 'anise', 'creosote', 'fishy', 'foul',
                  'musty', 'none', 'pungent', 'spicy')
      ),
      hash(
        keys   = c('a', 'd', 'f', 'n'),
        values = c('attached', 'descending', 'free', 'notched')
      ),
      hash(
        keys   = c('c', 'w', 'd'),
        values = c('close', 'crowded', 'distant')
      ),
      hash(
        keys   = c('b', 'n'),
        values = c('broad', 'narrow')
      ),
      colors,

```

```

    hash(
      keys   = c('e', 't'),
      values = c('enlarging', 'tapering')
    ),
    hash(
      keys   = c('b', 'c', 'u', 'e', 'z', 'r', '?'),
      values = c('bulbous', 'club', 'cup', 'equal', 'rhizomorphs',
                  'rooted', 'missing')
    ),
    surfaces,
    surfaces,
    colors,
    colors,
    hash(
      keys   = c('p', 'u'),
      values = c('partial', 'universal')
    ),
    colors,
    hash(
      keys   = c('n', 'o', 't'),
      values = c('none', 'one', 'two')
    ),
    hash(
      keys   = c('c', 'e', 'f', 'l', 'n', 'p', 's', 'z'),
      values = c('cobwebby', 'evanescent', 'flaring', 'large',
                  'none', 'pendant', 'sheathing', 'zone')
    ),
    colors,
    hash(
      keys   = c('a', 'c', 'n', 's', 'v', 'y'),
      values = c('abundant', 'clustered', 'numerous',
                  'scattered', 'several', 'solitary')
    ),
    hash(
      keys   = c('g', 'l', 'm', 'p', 'u', 'w', 'd'),
      values = c('grasses', 'leaves', 'meadows', 'paths',
                  'urban', 'waste', 'woods')
    )
  )
)
data |>
  rowwise() |>
  mutate(across(everything(), ~ values_map[[cur_column()]][[.]])) |>
  ungroup()
}

```

```
verbose_df <- make_legible(mushroom_df)
verbose_df |> head() |> select(poisonous, cap_shape, gill_color, odor, habitat)
```

```
## # A tibble: 6 x 5
##   poisonous cap_shape gill_color odor   habitat
##       <dbl> <chr>      <chr>   <chr>  <chr>
## 1         1 convex    black   pungent urban
## 2         0 convex    black   almond  grasses
## 3         0 bell      brown   anise   meadows
## 4         1 convex    brown   pungent urban
## 5         0 convex    black   none    grasses
## 6         0 convex    brown   almond  grasses
```

Data Analysis

This section is divided into Packaged Tools Analysis, using available tools such as `prcomp`, `nearZeroVariance`, `aov`, and more; and Custom Analysis, a `dplyr` look into data trends.

Packaged Tools Analysis

Here I use readily available tools to primarily determine which predictors are most influential and which can be omitted from analysis. Admittedly, I don't have a firm enough understanding of many of these tools to make confident conclusions; and am learning through usage.

Principal Component Analysis

First, I look for PCs that can be ignored.

```
pca <- prcomp(numeric_df)
which(cumsum(pca$sdev^2)/sum(pca$sdev^2) >= 0.90)
```

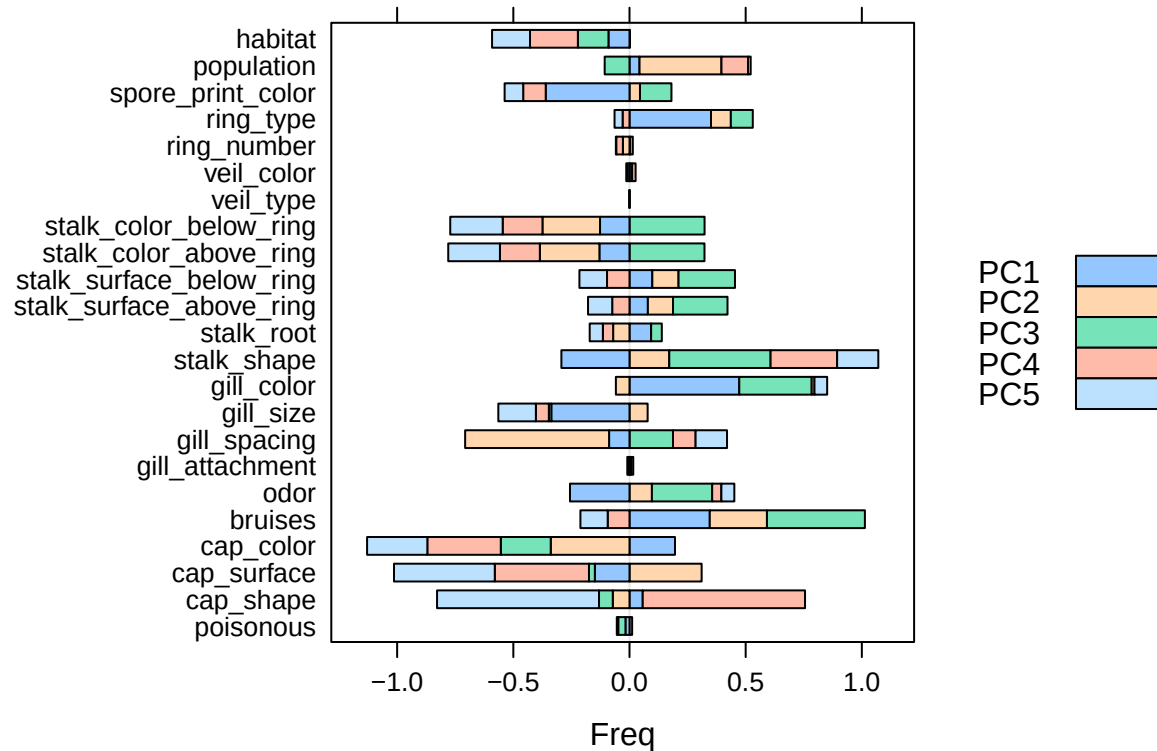
```
## [1] 11 12 13 14 15 16 17 18 19 20 21 22 23
```

Evidently the first 10 PCs explain 90% of the overall variation.

PC Rotation Barchart

Here, for clarity, I plot only the top 5 PCs - which, from `summary(pca)` are known to comprise about 19%, 13%, 12%, 11%, and 10% of the overall variance:

```
barchart(pca$rotation[,1:5], auto.key = TRUE)
```



To interpret this, I found it useful to also look at the output of `round(scale(pca$rotation[,1:5]),3)`; and I also read this conversation. The plot indicates that, for example, `gill_color`, `ring_type`, `bruises`, and `cap_color` are strongly and directly related to PC1, while `spore_print_color`, `gill_size`, `stalk_shape`, and `odor` are inversely related. We can also see that `veil_type` and `gill_attachment` have little sway, as they have only 1 and 2 distinct values, respectively.

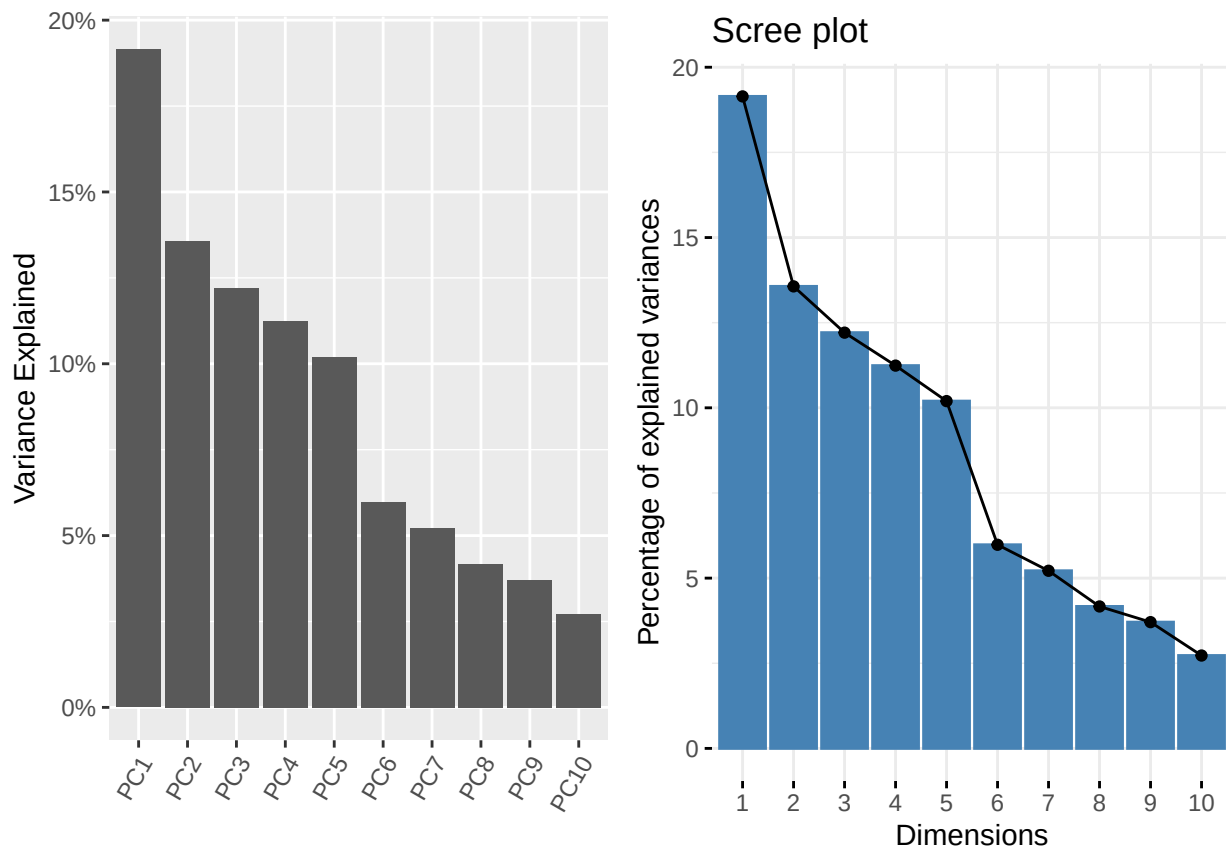
PC Variance Explained

And here is the %variance for the top 10 PCs, hand-rolled and via `factoextra::fviz_eig` - evidently known as a Scree plot:

```
s <- summary(pca)
p1 <- s$importance[,1:10] |>
  as.data.frame() |>
  rownames_to_column() |>
  filter(rowname == 'Proportion of Variance') |>
  pivot_longer(-rowname) |>
  select(-rowname) |>
  mutate(name = reorder(name, desc(value))) |>
  ggplot(aes(name, value)) +
  geom_col() +
  labs(x = '', y = 'Variance Explained') +
  theme(axis.text.x = element_text(angle = 60, vjust = 1, hjust = 1)) +
  scale_y_continuous(labels = scales::percent)

library(factoextra)
p2 <- fviz_eig(pca)

grid.arrange(p1, p2, ncol = 2)
```



It is evident that much of the variance is explained by the first five PCs, with significance declining thereafter.

PC Importance

Below are the column name, most/least correlating columns, and %importance for each PC explaining at least 11% of the total variance (11% chosen for PDF width):

```
sapply(which(s$importance[2,] > .11), function(i) {
  c(
    column      = names(numeric_df)[i],
    max         = names(numeric_df)[which.max(pca$rotation[,i])],
    min         = names(numeric_df)[which.min(pca$rotation[,i])],
    importance   = sprintf('%%.2f', s$importance[2,i]*100)
  )
})
```

##	PC1	PC2	PC3	PC4
## column	"poisonous"	"cap_shape"	"cap_surface"	"cap_color"
## max	"gill_color"	"population"	"stalk_shape"	"cap_shape"
## min	"spore_print_color"	"gill_spacing"	"cap_color"	"cap_surface"
## importance	"%19.14"	"%13.57"	"%12.21"	"%11.24"

Again also eyeballing the output of `round(scale(pca$rotation[,1:5]),3)`, the two are in agreement. For example, for PC1, `gill_color` is largest (2.242) and `spore_print_color` is smallest (-1.666).

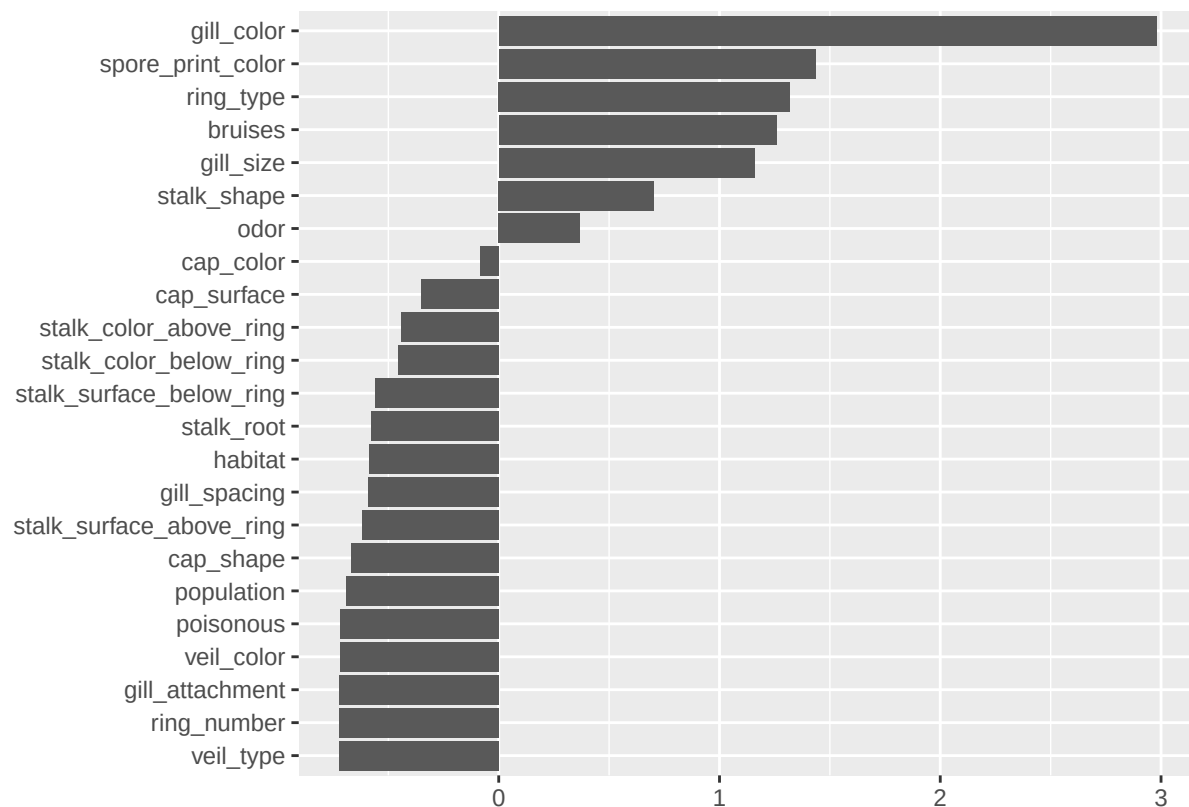
PC Variance Contribution

Here I plot the variance contribution of each column for PC1, via `prcomp` results and code derived from STHDA (Statistical Tools for High-Throughput Data Analysis):

```
var_coord_func <- function(loadings, comp.sdev){
  loadings*comp.sdev
}

loadings <- pca$rotation
sdev <- pca$sdev
var.coord <- t(apply(loadings, 1, var_coord_func, sdev))
var.cos2 <- var.coord^2
comp.cos2 <- apply(var.cos2, 2, sum)
contrib <- function(var.cos2, comp.cos2){var.cos2*100/comp.cos2}
var.contrib <- t(apply(var.cos2, 1, contrib, comp.cos2))

var.contrib[,1] |>
  as.data.frame() |>
  rownames_to_column(var = 'column') |>
  mutate(value = scale(pick(2)), column = reorder(column, value)) |>
  select(column, value) |>
  ggplot(aes(value, column)) + labs(x = '', y = '') +
  geom_col()
```



This does seem to illustrate the composition of PC1 clearly.

Near-Zero Variance

This indicates three columns have near-zero variance. (Assistance [here](#) and [here](#) for `cat` and `shQuote`.)

```
library(caret)
indices <- nearZeroVar(numeric_df)
columns <- names(numeric_df)[c(indices)]
cat(sprintf(
  "Indices: %s\nColumns: %s",
  paste(shQuote(indices), collapse=", "),
  paste(shQuote(columns), collapse=", ")
))
```

```
## Indices: '7', '17', '18'
```

```
## Columns: 'gill_attachment', 'veil_type', 'veil_color'
```

The latter two columns have already been deemed unnecessary via PCA. Column 7, `gill_attachment`, has the following distribution:

```
table(numeric_df[,7])
```

```
## gill_attachment
```

```
##      1      6
```

```
## 210 7914
```

From the `value_map` in Data Pre-Processing coupled with mapping letters to numbers, we can derive that “1” equals “a” (attached) and “6” equals “f” (free) - so 2.5% of gills are attached and 97.5% are free.

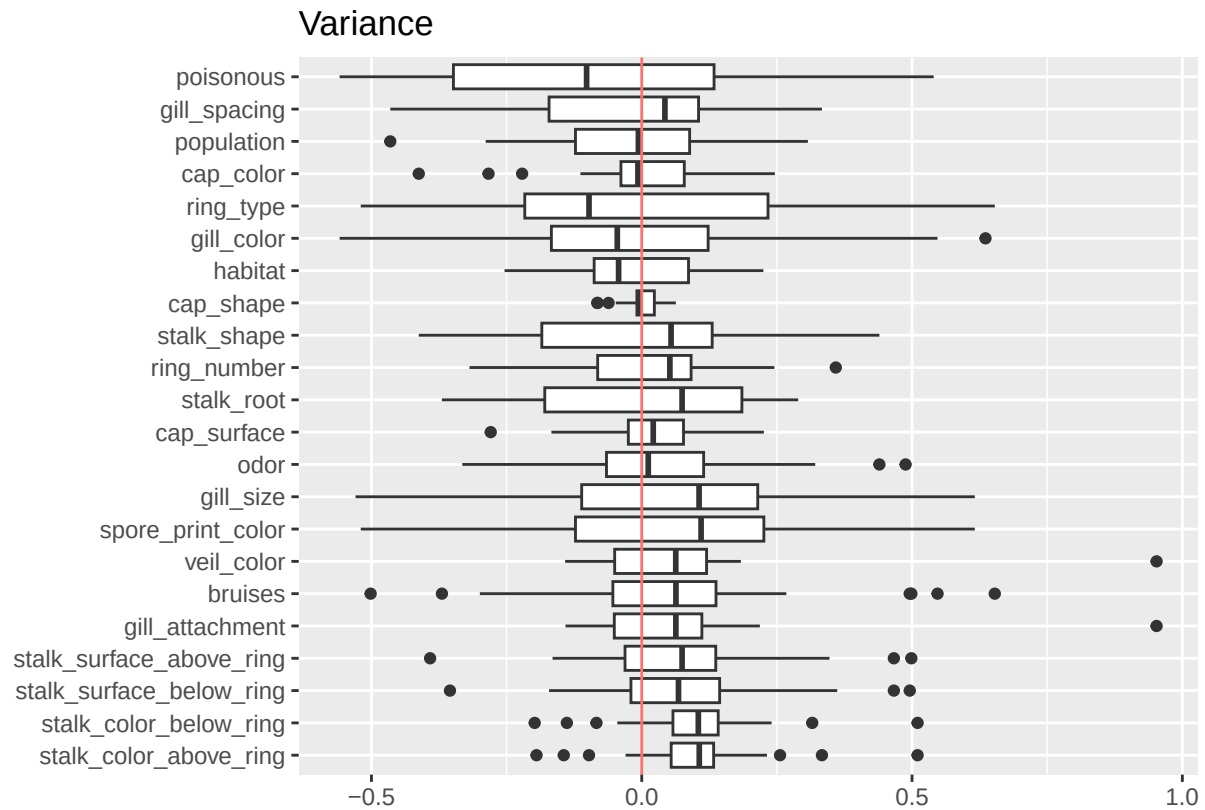
Variance, Covariance, Correlation

Here, I try boxplots of `var` and `cov`. I did `cor` too, but it looks identical to `var` so I've omitted it. I compute them first; but plot them separately as they do not arrange well with `grid.arrange`. (I also read this article as a refresher on variance, covariance and correlation.)

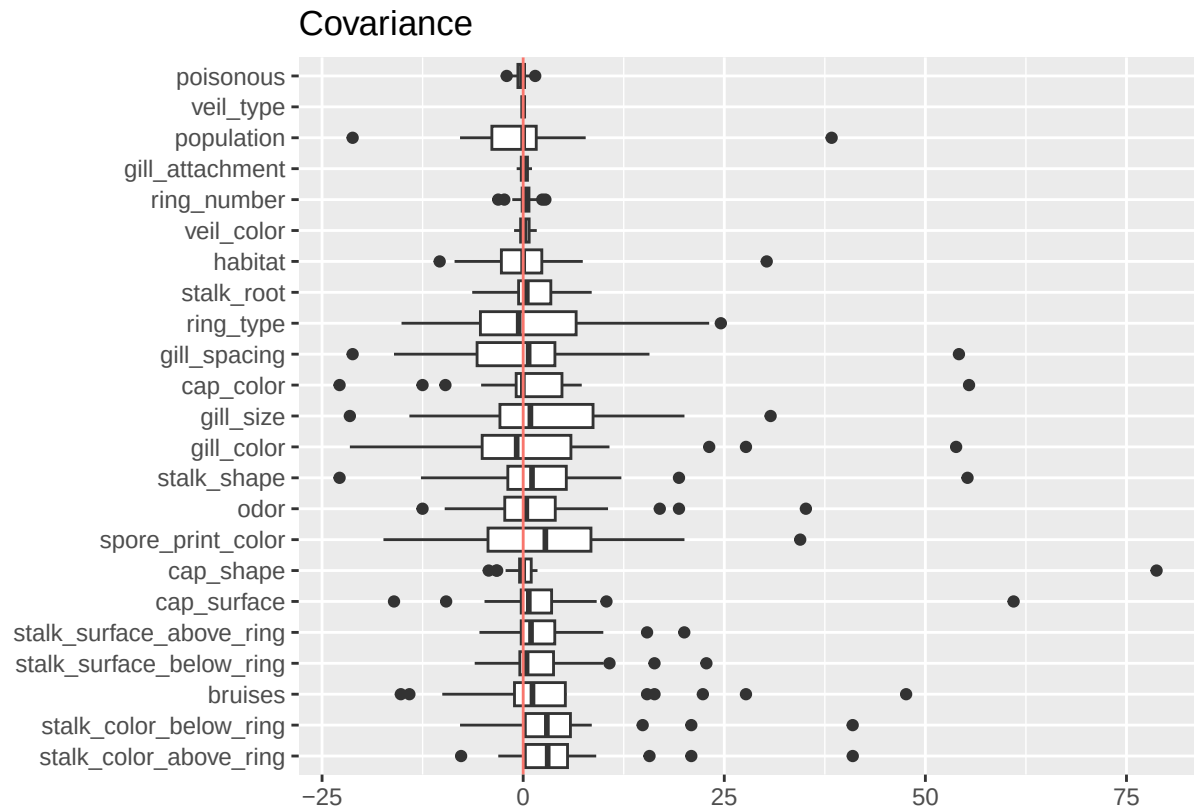
```
self_correlation <- 1
common_layers <- list(
  theme(legend.position = ''),
  geom_boxplot(),
  geom_hline(aes(yintercept = 0, color = 'pink')),
  coord_flip()
)

variance <- cor(numeric_df) |>
  as.data.frame() |>
  rownames_to_column() |>
  select(-rowname) |>
  pivot_longer(everything()) |>
  filter(value != self_correlation) |>
  group_by(name) |> mutate(avg = mean(value)) |> ungroup() |>
  mutate(name = reorder(name, desc(avg))) |>
  ggplot(aes(name, value)) +
  labs(x = '', y = '', title = 'Variance') +
  common_layers

covariance <- cov(numeric_df) |>
  as.data.frame() |>
  rownames_to_column() |>
  select(-rowname) |>
  pivot_longer(everything()) |>
  filter(value != self_correlation) |>
  group_by(name) |> mutate(avg = mean(value)) |> ungroup() |>
  mutate(name = reorder(name, desc(avg))) |>
  ggplot(aes(name, value)) +
  labs(x = '', y = '', title = 'Covariance') +
  common_layers
```



From this, I would guess that `poisonous` has primarily *negative* correlations with its predictors, with its mean below zero; and that `stalk_color_*` have most definitive correlations, with their inter-quartile ranges completely above zero. The outliers are about evenly split above- and below zero.

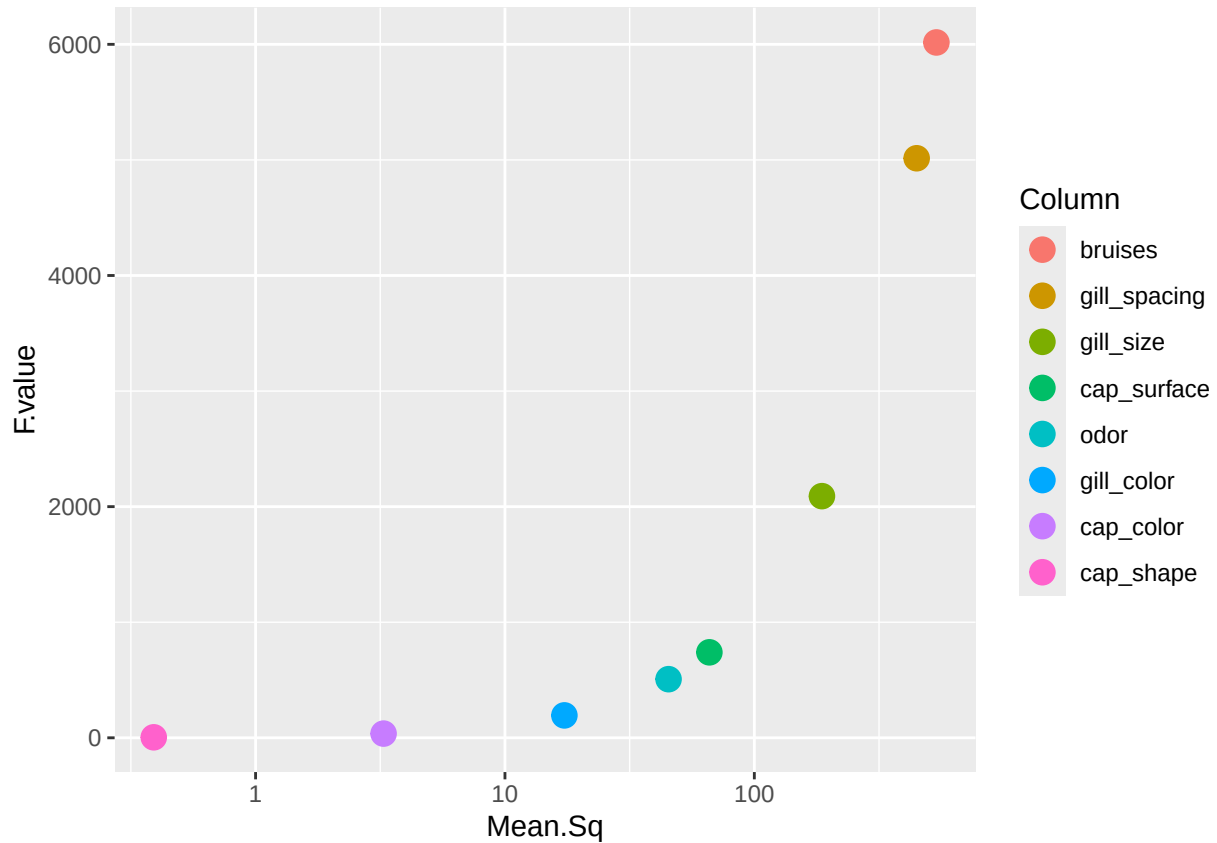


Note that `veil_type` was NA in Variance but joins the mix here, as a covariant. The outlier ratio shifted - many more are now above zero. And in addition to `stalk_color_*`, `stalk_surface_*` and `cap_surface` are also almost wholly above zero.

ANOVA (aov)

aov suggests that bruises, gill_spacing, gill_size are most influential.

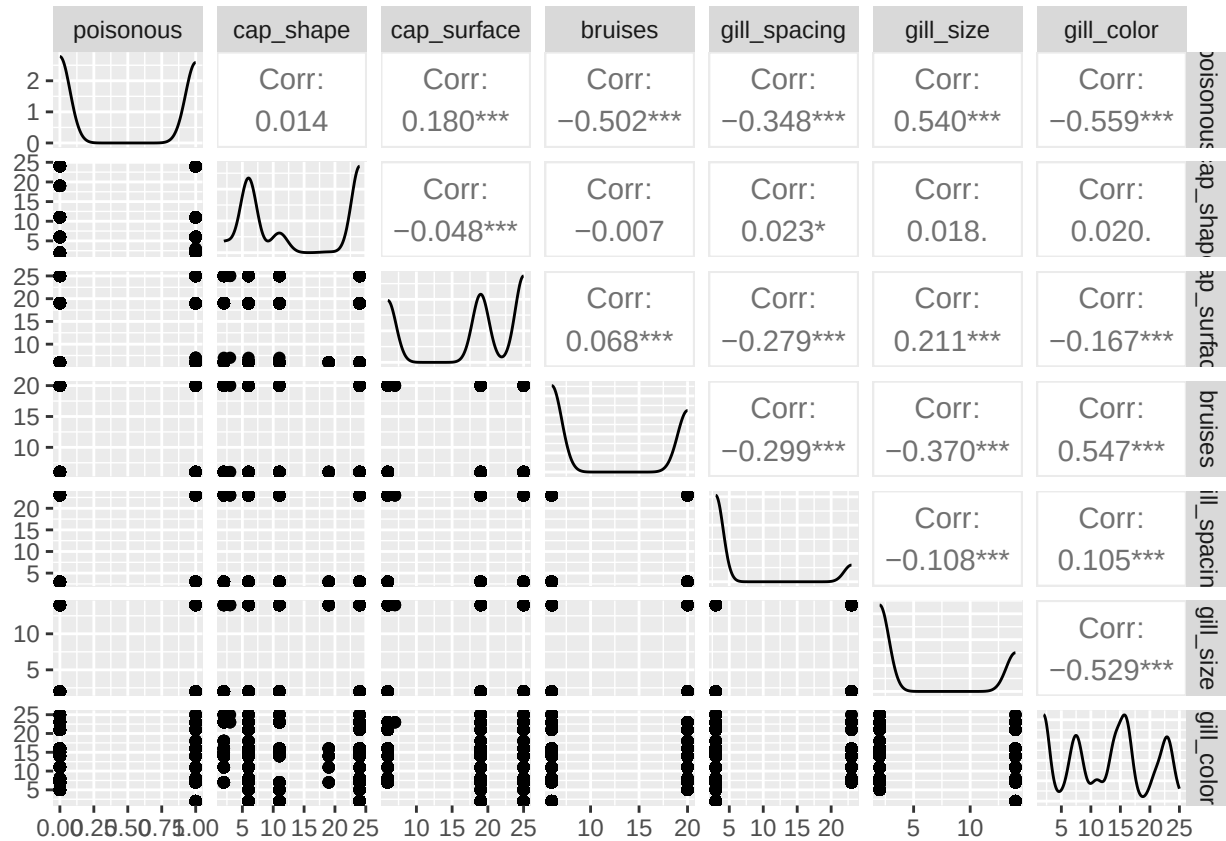
```
unclass(summary(stats::aov(poisonous ~ ., data = numeric_df[c(1:6,8:10)]))) |>
  as.data.frame() |>
  rownames_to_column(var = 'Column') |>
  filter(!is.na(F.value)) |>
  mutate(Column = reorder(Column, desc(F.value))) |>
  ggplot(aes(Mean.Sq, F.value, color = Column)) +
  geom_point(cex=4) +
  scale_x_log10()
```



GGPAIRS

Here I plot the correlations for what seem to be the most influential predictors. (The plot becomes unwieldy with too many predictors.) I'm still learning about this tool, not ready to make any conclusions from it yet.

```
library(GGally)
ggpairs(numeric_df[, c(1:3,5,8:10)])
```



Packaged Tools Conclusion

From a variety of packaged tools, I conclude that three columns can be omitted due to near-zero variation and 13 PCs can be omitted as the other 10 explain 90% of the variation. If I end up using **train**, at least **pca** and **nzv** should be worthwhile pre-options, with experimental tuning. I also have some sense of the more influential columns, which can be a starting point for subsequent analysis.

Custom Analysis

Here I inspect the data with SQL-like `dplyr` methods, looking for telltale patterns.

First, I define three layers that will be used verbatim throughout the plots:

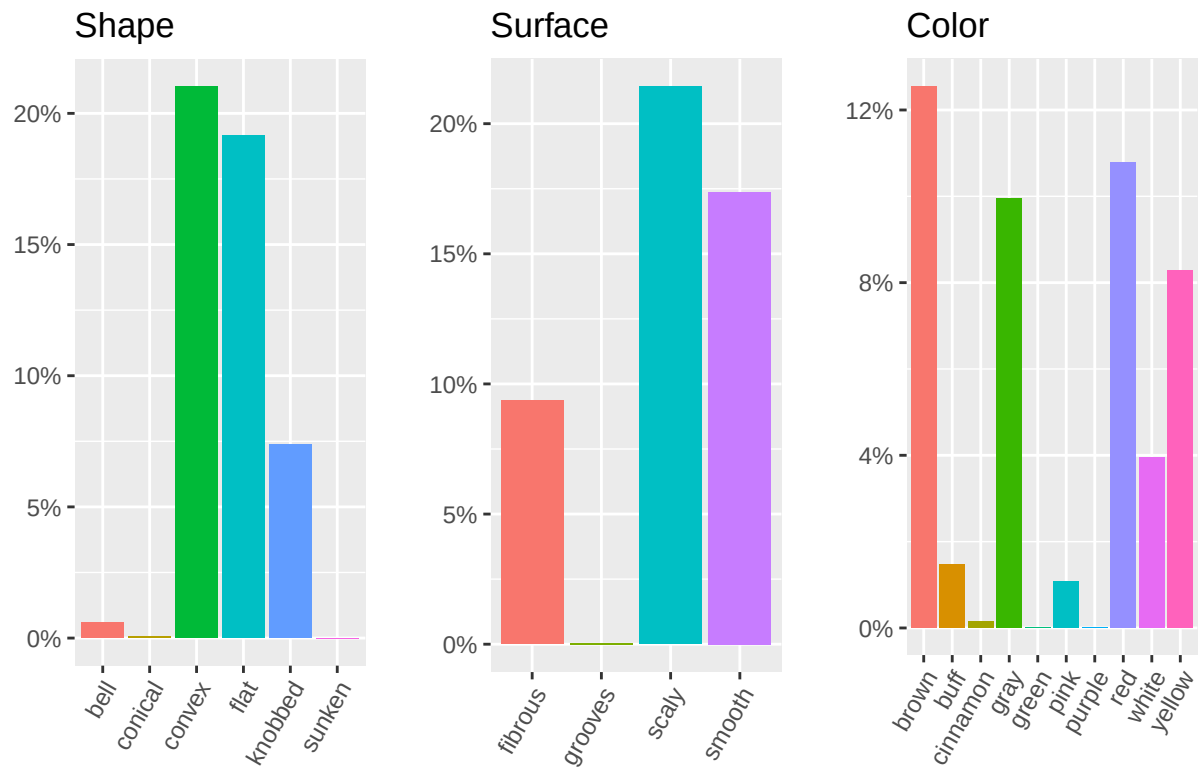
```
common_layers <- list(  
  geom_bar(stat = 'identity'),  
  scale_y_continuous(labels = scales::percent),  
  theme(  
    legend.position = '',  
    axis.text.x = element_text(angle = 60, vjust = 1, hjust = 1)  
  )  
)
```

Mushroom Cap Attributes

Here I make bar plots of the poisonous percentage for each column value of the cap-related columns.

```
cap_shape <- aggregate(verbose_df, poisonous ~ cap_shape, sum) |>  
  mutate(poisonous = poisonous/nrow(verbose_df)) |>  
  ggplot(aes(cap_shape, poisonous, fill = cap_shape)) +  
  labs(x = "", y = "", title = "Shape") +  
  common_layers  
  
cap_surface <- aggregate(verbose_df, poisonous ~ cap_surface, sum) |>  
  mutate(poisonous = poisonous/nrow(verbose_df)) |>  
  ggplot(aes(cap_surface, poisonous, fill = cap_surface)) +  
  labs(x = "", y = "", title = "Surface") +  
  common_layers  
  
cap_color <- aggregate(verbose_df, poisonous ~ cap_color, sum) |>  
  mutate(poisonous = poisonous/nrow(verbose_df)) |>  
  ggplot(aes(cap_color, poisonous, fill = cap_color)) +  
  labs(x = "", y = "", title = "Color") +  
  common_layers  
  
grid.arrange(  
  cap_shape, cap_surface, cap_color,  
  ncol = 3,  
  top = textGrob(  
    "Poison Percentage by Cap Attributes",  
    gp = gpar(fontsize = 12, font = 3)  
  )  
)
```

Poison Percentage by Cap Attributes



Mushroom Gill Attributes

Here, I do the same for gill-related columns. Note, `gill_attachment` has been shown to have near-zero variance, and will likely be omitted in final analysis.

```
gill_attachment <- aggregate(verbose_df, poisonous ~ gill_attachment, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(gill_attachment, poisonous, fill = gill_attachment)) +
  labs(x = "", y = "", title = "Attachment") +
  common_layers

gill_spacing <- aggregate(verbose_df, poisonous ~ gill_spacing, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(gill_spacing, poisonous, fill = gill_spacing)) +
  labs(x = "", y = "", title = "Spacing") +
  common_layers

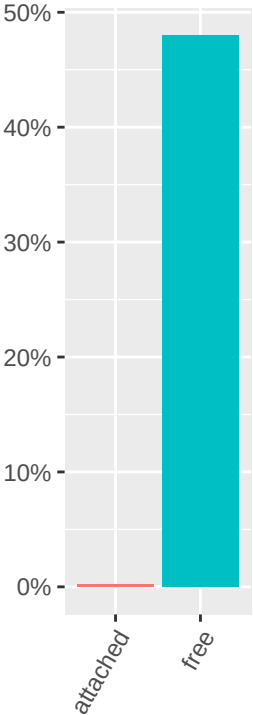
gill_size <- aggregate(verbose_df, poisonous ~ gill_size, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(gill_size, poisonous, fill = gill_size)) +
  labs(x = "", y = "", title = "Size") +
  common_layers

gill_color <- aggregate(verbose_df, poisonous ~ gill_color, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(gill_color, poisonous, fill = gill_color)) +
  labs(x = "", y = "", title = "Color") +
  common_layers

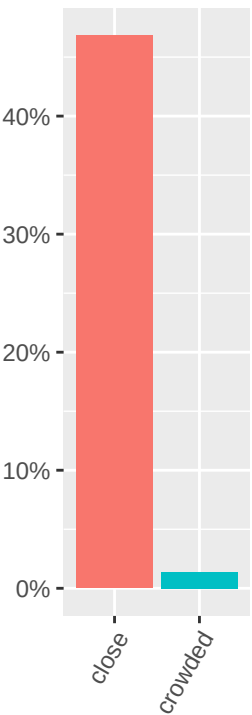
grid.arrange(
  gill_attachment, gill_spacing, gill_size, gill_color,
  ncol = 4,
  top = textGrob(
    "Poison Percentage by Gill Attributes",
    gp = gpar(fontsize = 12, font = 3)
  )
)
```

Poison Percentage by Gill Attributes

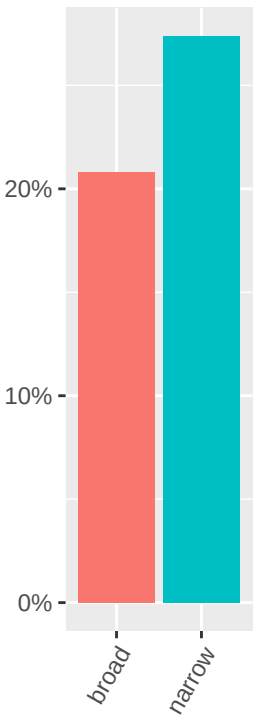
Attachment



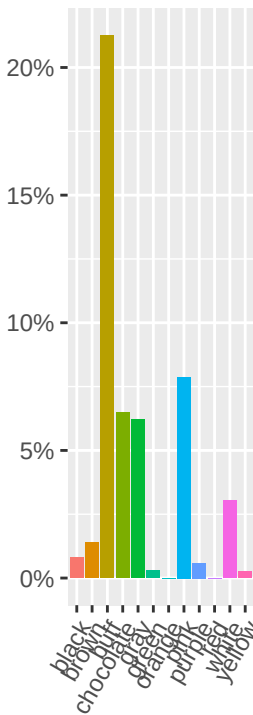
Spacing



Size



Color



Mushroom Stalk Attributes

Here, I do the same for stalk-related columns.

```
stalk_shape <- aggregate(verbose_df, poisonous ~ stalk_shape, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(stalk_shape, poisonous, fill = stalk_shape)) +
  labs(x = "", y = "", title = "Shape") +
  common_layers

stalk_root <- aggregate(verbose_df, poisonous ~ stalk_root, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(stalk_root, poisonous, fill = stalk_root)) +
  labs(x = "", y = "", title = "Root") +
  common_layers

stalk_surface_above <- aggregate(verbose_df, poisonous ~ stalk_surface_above_ring, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(stalk_surface_above_ring, poisonous, fill = stalk_surface_above_ring)) +
  labs(x = "", y = "", title = "Surface", subtitle = "Above Ring") +
  common_layers

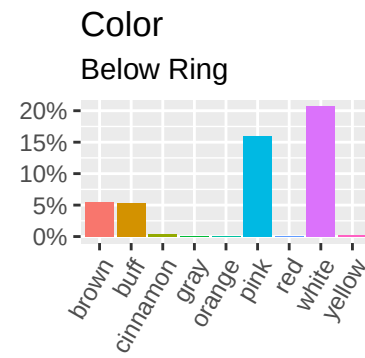
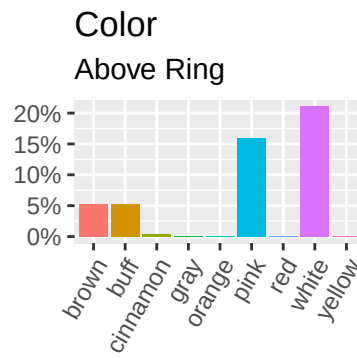
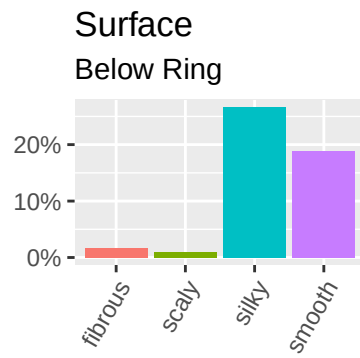
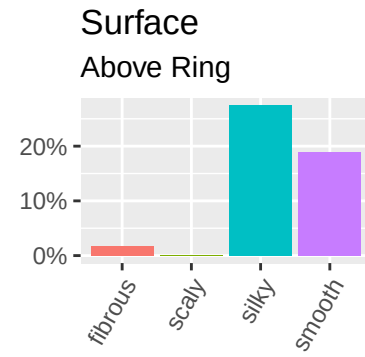
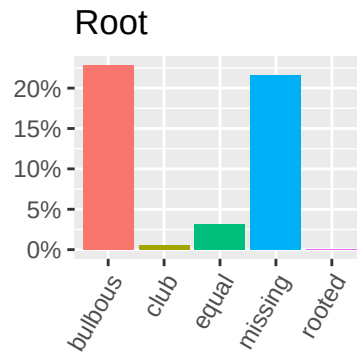
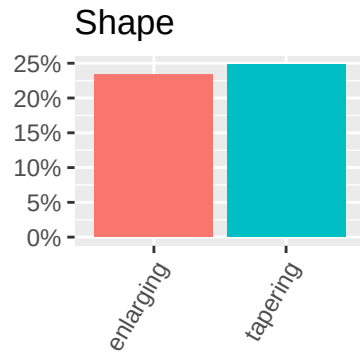
stalk_surface_below <- aggregate(verbose_df, poisonous ~ stalk_surface_below_ring, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(stalk_surface_below_ring, poisonous, fill = stalk_surface_below_ring)) +
  labs(x = "", y = "", title = "Surface", subtitle = "Below Ring") +
  common_layers

stalk_color_above <- aggregate(verbose_df, poisonous ~ stalk_color_above_ring, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(stalk_color_above_ring, poisonous, fill = stalk_color_above_ring)) +
  labs(x = "", y = "", title = "Color", subtitle = "Above Ring") +
  common_layers

stalk_color_below <- aggregate(verbose_df, poisonous ~ stalk_color_below_ring, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(stalk_color_below_ring, poisonous, fill = stalk_color_below_ring)) +
  labs(x = "", y = "", title = "Color", subtitle = "Below Ring") +
  common_layers

grid.arrange(
  stalk_shape, stalk_root, stalk_surface_above,
  stalk_surface_below, stalk_color_above, stalk_color_below,
  ncol = 3,
  nrow = 2,
  top = textGrob(
    "Poison Percentage by Stalk Attributes",
    gp = gpar(fontsize = 12, font = 3)
  )
)
```

Poison Percentage by Stalk Attributes



Mushroom Veil and Ring Attributes

Here I do the same for veil- and ring-related columns. Note, `veil_type` and `veil_color` have been shown to have near-zero variance and will likely be omitted from final analysis.

```
veil_type <- aggregate(verbose_df, poisonous ~ veil_type, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(veil_type, poisonous, fill = veil_type)) +
  labs(x = "", y = "", title = "Veil Type") +
  common_layers

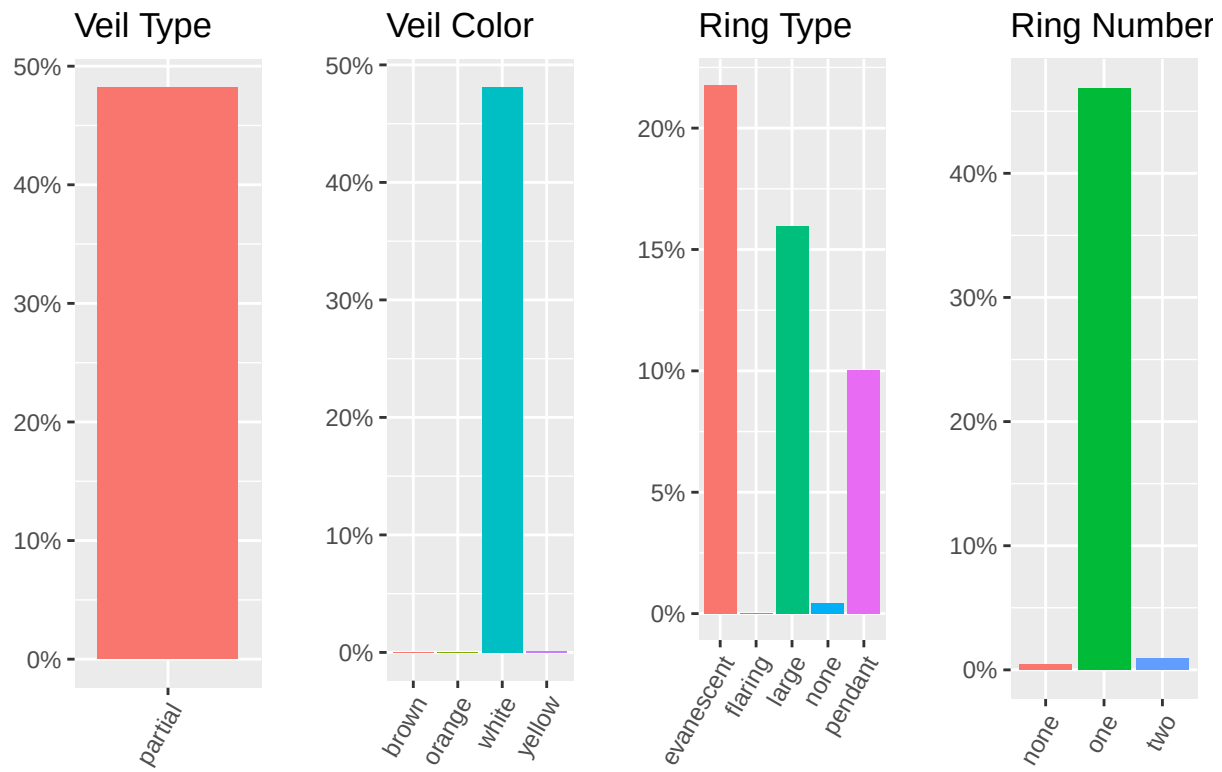
veil_color <- aggregate(verbose_df, poisonous ~ veil_color, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(veil_color, poisonous, fill = veil_color)) +
  labs(x = "", y = "", title = "Veil Color") +
  common_layers

ring_type <- aggregate(verbose_df, poisonous ~ ring_type, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(ring_type, poisonous, fill = ring_type)) +
  labs(x = "", y = "", title = "Ring Type") +
  common_layers

ring_number <- aggregate(verbose_df, poisonous ~ ring_number, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(ring_number, poisonous, fill = ring_number)) +
  labs(x = "", y = "", title = "Ring Number") +
  common_layers

grid.arrange(
  veil_type, veil_color, ring_type, ring_number,
  ncol = 4,
  top = textGrob(
    "Poison Percentage by Veil and Ring Attributes",
    gp = gpar(fontsize = 12, font = 3)
  )
)
```

Poison Percentage by Veil and Ring Attributes



Mushroom Other Attributes

Here I do the same for a variety of one-off columns.

```
bruises <- aggregate(verbose_df, poisonous ~ bruises, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(bruises, poisonous, fill = bruises)) +
  labs(x = "", y = "", title = "Bruises") +
  common_layers

odor <- aggregate(verbose_df, poisonous ~ odor, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(odor, poisonous, fill = odor)) +
  labs(x = "", y = "", title = "Odor") +
  common_layers

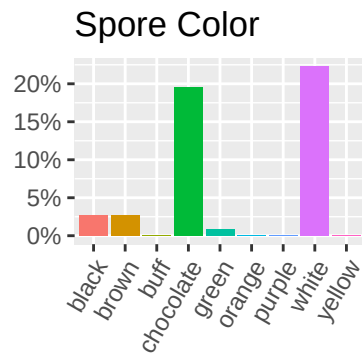
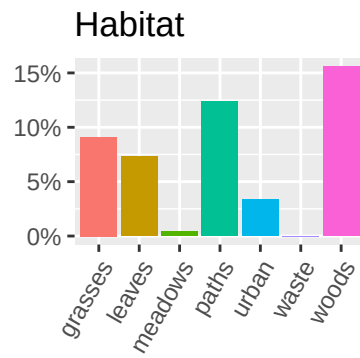
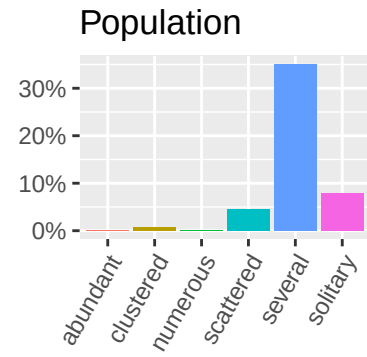
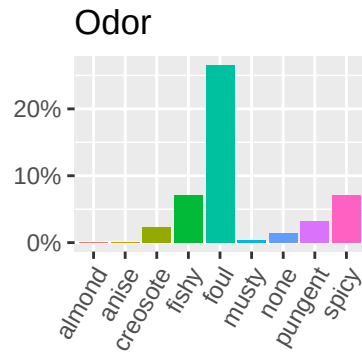
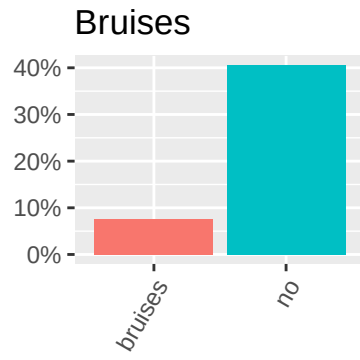
population <- aggregate(verbose_df, poisonous ~ population, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(population, poisonous, fill = population)) +
  labs(x = "", y = "", title = "Population") +
  common_layers

habitat <- aggregate(verbose_df, poisonous ~ habitat, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(habitat, poisonous, fill = habitat)) +
  labs(x = "", y = "", title = "Habitat") +
  common_layers

spore_print_color <- aggregate(verbose_df, poisonous ~ spore_print_color, sum) |>
  mutate(poisonous = poisonous/nrow(verbose_df)) |>
  ggplot(aes(spore_print_color, poisonous, fill = spore_print_color)) +
  labs(x = "", y = "", title = "Spore Color") +
  common_layers

grid.arrange(
  bruises, odor, population, habitat, spore_print_color,
  ncol = 3,
  top = textGrob(
    "Poison Percentage by Other Attributes",
    gp = gpar(fontsize = 12, font = 3)
  )
)
```

Poison Percentage by Other Attributes



Custom Analysis Conclusion

The above research provides some direction. Here I calculate poison percentage for promising patterns.

```
verbose_df |>
  filter(
    gill_color == 'buff'
    # odor %in% c('fishy', 'foul', 'spicy')
    # etc.
  ) |> summarize(n=n(), pct = sum(poisonous)/n)
```

```
## # A tibble: 1 x 2
##       n    pct
##   <int> <dbl>
## 1  1728     1
```

And here are the patterns I tried, and the number of rows returned and percentage of poisonous:

Table 1: Poison Tendencies

ATTRIBUTE_SET	NUMBER	POISON_PCT
Cap - convex/flat; scaly/smooth/fibrous; brown/gray/red/yellow	5690	48.7
Gills - buff-colored (*)	1728	100.0
Stalk - silky/smooth; white/pink/brown/buff	6648	56.1
Ring - one; evanescent/large/pendant	7440	51.2
Root - bulbous/missing	6256	57.8
Bruises - no	4748	69.3
Odor - fishy/foul/spicy (*)	3312	100.0
Population - several/solitary/scattered	7000	55.2
Habitat - grasses/leaves/paths/woods	7272	49.6
Spore Color - white/chocolate	4020	84.5

* Note: Buff-colored gills and fishy/foul/spicy odor overlap

Results

First Model

I pieced together some of the conditions noted in Custom Analysis and filtered *out* matching rows; then iteratively looked for patterns in the remaining poisonous rows and added them to the filtering; etc. until arriving at this:

```
verbose_df |>
  mutate(
    guess = ifelse((
      odor %in% c('fishy', 'foul', 'spicy', 'creosote', 'pungent')
      | ring_number == 'none'
      | cap_shape == 'conical'
      | spore_print_color == 'green'
    ), 1, 0)
  ) |>
  summarize(accuracy= sum(guess==poisonous)/n()) |>
  pull(accuracy)
```

```
## [1] 0.9945839
```

That seems too easy and very accurate. But the webpage indicates that I should be near 100% - and I did not spot any pattern in the balance of poisonous rows that do not match this query.

Second Model

During the first project, I spent a lot of time with `train` and various algorithms, only to sidestep the work because the data was too large. Now I'd like to revisit; and `ranger` was one I spent most time on.

```
index <- createDataPartition(y = numeric_df$poisonous, times = 1, p = 0.5, list = FALSE)
train <- numeric_df[index,]
test  <- numeric_df[-index,]
fit    <- ranger(factor(poisonous) ~ ., train)
pred   <- predict(fit, test)
cm     <- confusionMatrix(pred$predictions, factor(test$poisonous))

sprintf('Accuracy: %%.2f', cm$overall[['Accuracy']]*100)
```

```
## [1] "Accuracy: %100.00"
```

Final Model

What?! No tuning?

All that's left to do is run it against the full set:

```
fit <- ranger(factor(poisonous) ~ ., numeric_df)
pred <- predict(fit, numeric_df)
cm <- confusionMatrix(pred$predictions, factor(numeric_df$poisonous))

sprintf('Accuracy: %%.2f', cm$overall[['Accuracy']]*100)

## [1] "Accuracy: %100.00"
```

Conclusion

I envisioned laboriously fine-tuning **ranger** or other algorithm to reach a decent score. This was actually disappointing; below is the brief progression of models:

Table 2: Model Progression

MODEL	ACCURACY
first	0.9945839
second	1.0000000
final	1.0000000

Overall, I did not learn much with this project other than additional R skills - particularly:

- **aggregate** and **rowwise**, coupled with **mutate** and **across**
- adding titles to **grid.arrange** and styling to **kable**
- **hash**, a fundamental feature in Perl

I also carried over two lessons from the first project - to NOT use Rmd markup tables, which do not **knit** well, nor headings below 3, which do not **knit** to my taste.

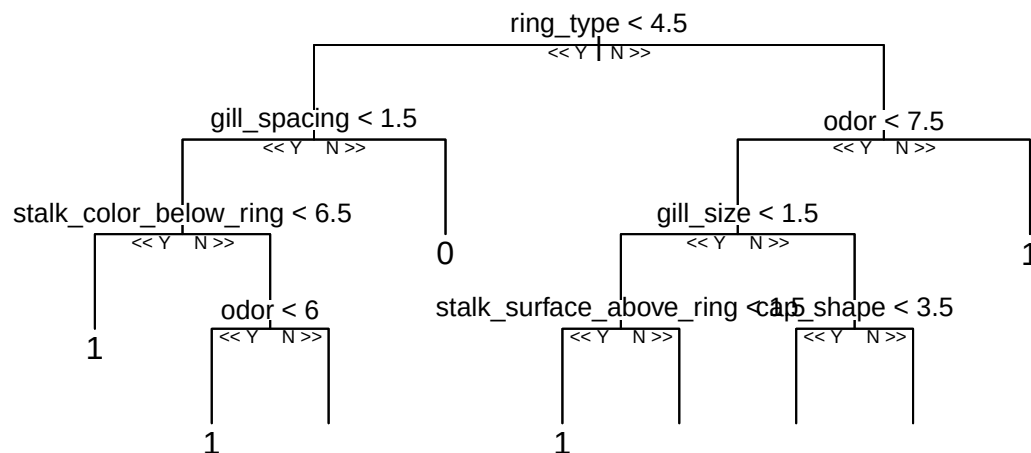
Decision Tree

Finally, it occurred to me that if I had solved this immediately with out-of-the-box **ranger** without first taking the **dplyr** approach, I would have no idea how the algorithm might make its determinations - i.e., what its decision tree might resemble. With that in mind, I found this about **randomForest** and tried it. Note, I set **depth** for legibility, at the expense of not reaching all leaf nodes.

```
library(devtools)
install_github('munoztd0/reprtree')
eval(substitute(library(pkg), list(pkg='reprtree'))))

library(randomForest)
library(reprtree)
fit <- randomForest(factor(poisonous) ~ ., verbose_df)
cm <- confusionMatrix(pred$predictions, factor(numeric_df$poisonous))
plot.getTree(
  fit,
  k      = 1,
  depth  = 5,
  main   = sprintf(
    '%s (%%%.2f accuracy)',
    'RandomForest Decision Tree',
    cm$overall[['Accuracy']]*100
  )
)
```

RandomForest Decision Tree (%100.00 accuracy)



References

<https://dplyr.tidyverse.org/articles/rowwise.html>

<https://www.benjaminbell.co.uk/2018/03/principal-components-analysis-pca-in-r-part-2.html>

<https://www.modernstatisticswithr.com/regression.html>

<https://stackoverflow.com/questions/14726078/changing-title-in-multiplot-ggplot2-using-grid-arrange>

<https://www.sthda.com/english/articles/31-principal-component-methods-in-r-practical-guide/118-principal-component-analysis-in-r-prcomp-vs-princomp>

https://haozhu233.github.io/kableExtra/awesome_table_in_pdf.pdf

<https://stats.stackexchange.com/questions/41443/how-to-actually-plot-a-sample-tree-from-randomforest-gettree>

<https://stats.stackexchange.com/questions/144508/conclusions-from-output-of-a-principal-component-analysis>

<https://stackoverflow.com/questions/18747091/reuse-ggplot-layers-in-multiple-plots/44721060>

<https://www.countbayesie.com/blog/2015/2/21/variance-co-variance-and-correlation>

<https://stackoverflow.com/questions/6347356/creating-a-comma-separated-vector>

<https://stackoverflow.com/questions/28757372/printing-line-breaks-using-sprintf-with-shiny>

Appendix

While the movielens project was graded by peers, this project was graded by staff. Below is their feedback and my subsequent work to address it.

Staff Feedback

Staff response was constructive, positive with suggestions (to which I added numbers).

This data science capstone project demonstrates strong exploratory data analysis and visualization work. You demonstrate impressive R programming skills, effectively transforming categorical mushroom data, and creating comprehensive and clear visualizations across all mushroom attributes. Good understanding of various R packages and statistical techniques, progressing logically from simple pattern recognition to more sophisticated modeling approaches using randomForest algorithms.

There are some areas for enhancement. Notably, the project (1) lacks acknowledgment of class imbalance despite achieving 100% accuracy, which is a strong indicator of imbalanced data that should be explicitly examined and reported. You want to provide class distribution counts and (2) consider metrics beyond accuracy (precision, recall, F1-score). (3) Data scaling was not applied before PCA analysis, which is essential for distance-based methods. (4) You want to establish simpler baselines like logistic regression, which would be appropriate for this binary classification problem. (5) The project lacks specific limitations discussion and future work recommendations.

Response

1. *Acknowledgement of class imbalance.*

The many bar plots indicate a near-even balance of the two classes, but I should have been explicit.

```
# prop.table(table(numeric_df$poisonous)) # simpler, but less PDF-friendly
numeric_df |>
  summarize(Poisonous = sum(poisonous)/n(), Harmless = 1-Poisonous) |>
  knitr::kable(caption = 'Poisonous Distribution') |>
  kable_styling()
```

Table 3: Poisonous Distribution

Poisonous	Harmless
0.4820286	0.5179714

2. Consider metrics beyond accuracy.

I thought 100% accuracy was enough if the classes are reasonably balanced. Here are other metrics:

```
metrics <- c('Sensitivity', 'Specificity', 'Prevalence', 'Recall', 'Precision')
fit <- ranger(factor(poisonous) ~ ., numeric_df)
pred <- predict(fit, numeric_df)
cm <- confusionMatrix(pred$predictions, factor(numeric_df$poisonous))
round(c(
  cm$byClass[metrics],
  F1 = caret::F_meas(
    data = factor(pred$predictions),
    reference = factor(numeric_df$poisonous)
  )
),3) |>
  as.data.frame() |>
  rownames_to_column() |>
  pivot_wider(names_from = rowname, values_from = c(2)) |>
  knitr::kable(caption = 'Ranger Classification Metrics') |> kable_styling()
```

Table 4: Ranger Classification Metrics

Sensitivity	Specificity	Prevalence	Recall	Precision	F1
1	1	0.518	1	1	1

3. Data scaling for PCA.

Fair point; though here it seems to have a punitive effect, with more PCs needed to explain 90% variance? To scale, `veil_type` must be omitted or `prcomp` complains, “cannot rescale a constant/zero column to unit variance”:

```
columns <- c(1:16,18:23)
pca <- prcomp(numeric_df[,columns], scale. = FALSE)
unscaled <- which(cumsum(pca$sdev^2)/sum(pca$sdev^2) < 0.90)
pca <- prcomp(numeric_df[,columns], scale. = TRUE)
scaled <- which(cumsum(pca$sdev^2)/sum(pca$sdev^2) < 0.90)

knitr::kable(
  tibble(
    SCALING = c('unscaled', 'scaled'),
    'PCs Explaining 90% Variance' = c(
      paste(unscaled, collapse = ', '),
      paste(scaled, collapse = ', ')
    )
  ),
  caption = 'Data Scaling for PCA'
) |> kable_styling()
```

Table 5: Data Scaling for PCA

SCALING	PCs Explaining 90% Variance
unscaled	1, 2, 3, 4, 5, 6, 7, 8, 9, 10
scaled	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12

4. *Simpler baselines like logistic regression.*

Simpler than querying with `dplyr`? And wait, regression? Surely this is a classification case, with two discrete outcomes? OK...

```
fit    <- glm(poisonous ~ ., data = numeric_df, family = 'binomial')
pred   <- predict(fit, numeric_df)
y_hat  <- factor(ifelse(pred < .5, 0, 1))
cm     <- confusionMatrix(y_hat, factor(numeric_df$poisonous))

round(c(
  cm$byClass[metrics],
  F1 = caret::F_meas(
    data      = y_hat,
    reference = factor(numeric_df$poisonous)
  )
),3) |>
  as.data.frame() |>
  rownames_to_column() |>
  pivot_wider(names_from = rowname, values_from = c(2)) |>
  knitr::kable(caption = 'Logistic Regression with GLM') |> kable_styling()
```

Table 6: Logistic Regression with GLM

Sensitivity	Specificity	Prevalence	Recall	Precision	F1
0.972	0.944	0.518	0.972	0.949	0.961

5. *Lacks discussion of limitations and future work considerations.*

Given 100% accuracy plus all the other affirmative metrics of the final solution, I do not know what to say other than that I see no limitations or future work considerations.